MIDTERM PRACTICAL PRACTICE

Student Supplies Ordering System

I. General Instructions

Develop a functional Student Supplies Ordering System for a small school supplies shop near the campus. The application must allow customers to select school supplies, manage quantities, calculate the total amount, enter their cash payment, and receive a digital receipt.

You may use any programming language, framework, or AI-assisted development tool discussed in class.

- This examination must be completed individually.
- You may use AI tools to generate, debug, and improve your code.
- You must understand and be able to explain the code generated with AI assistance.
- You must use Git and GitHub to track your development progress.
- A database and user authentication are not required.
- Prioritize a working application over advanced features or complicated designs.

II. Scenario and Product List

A small school supplies shop near the campus wants to improve its manual ordering process. Develop a desktop-friendly or browser-based application that allows customers to purchase school supplies conveniently.

Product	Price
Ballpen	₱12.00
Pencil	₱10.00
Notebook	₱35.00
Yellow Pad	₱45.00
Eraser	₱8.00
Ruler	₱15.00

III. Step-by-Step Application Development

Step 1: Set Up Your Project

1. Open your preferred code editor or development environment.
2. Create a project named StudentSuppliesOrderingSystem.
3. Set up the files required by your chosen programming language or framework.
4. Run the project and confirm that the application starts successfully.
5. Create a simple interface with the following sections:
    - Product List
    - Shopping Cart
    - Order Summary
    - Payment Section
    - Receipt Section

Step 2: Display the Products

1. Display all six school supplies and their prices.
2. Add an **Add to Cart** button for each product.
3. Ensure that the product names and prices are readable.
4. Use hard-coded product information. A database is not necessary.

Step 3: Implement the Shopping Cart

1. Allow customers to add products to the cart.
2. Display each selected product's name, price, quantity, and subtotal.
3. Add buttons to increase and decrease the quantity.
4. Prevent the quantity from becoming zero or negative. If the quantity reaches zero, remove the item or provide a separate remove button.
5. Allow customers to remove an item from the cart.
6. Automatically update the order summary whenever the cart changes.

Formula:

Subtotal = Unit Price × Quantity

Total Amount = Sum of All Item Subtotals

Step 4: Implement Cash Payment

1. Add a numeric input field for the amount paid by the customer.
2. Add a **Confirm Payment** button.
3. Validate the entered payment amount.
4. Display an appropriate error message when:
    - The payment field is empty.

    - The payment is not a valid number.
    - The payment is negative.
    - The payment is less than the total amount due.
5. If the payment is sufficient, compute the change.

Formula:

Change = Amount Paid − Total Amount

Step 5: Generate the Digital Receipt

After successful payment, display a receipt containing:

- Transaction reference number
- Date and time, if available
- Purchased products
- Quantity of each product
- Unit price and subtotal
- Total amount due
- Amount paid
- Change
- Payment confirmation message

Add a **New Transaction** button that clears the previous cart, payment amount, validation messages, and receipt.

Step 6: Test Your Application

Perform the following tests before submitting:

1. Add one product to the cart.
2. Add multiple products to the cart.
3. Increase and decrease product quantities.
4. Remove a product from the cart.
5. Check whether the total updates correctly.
6. Enter a blank payment amount.
7. Enter a negative payment amount.
8. Enter an amount lower than the total.
9. Enter a sufficient payment amount and verify the change.
10. Complete a transaction and start a new one.

Correct any errors you encounter.

IV. Step-by-Step Git and GitHub Requirements

You must use Git and GitHub throughout the development process. **Do not wait until the end of the examination to upload your entire project in one commit.**

Step 1: Create a GitHub Account

1. Visit https://github.com.
2. Sign in to your account or create one if you do not have an account.
3. Ensure that you can access your GitHub dashboard.

Step 2: Create a Repository

1. Click the **+** icon or the **New repository** option.
2. Enter the repository name: StudentSuppliesOrderingSystem.
3. Add the description: IT415 Midterm Practical Examination - Student Supplies Ordering System.
4. Set the repository visibility to **Public**, unless instructed otherwise.
5. You may initialize the repository with a README if you know how to clone and work with an existing repository. Otherwise, create an empty repository to simplify the initial setup.
6. Click **Create repository**.
7. Copy the repository URL. You will use this URL to connect your local project to GitHub.

Step 3: Initialize Git in Your Project

Open the terminal in your project folder.

If you created an empty GitHub repository, execute the following commands:

```
git init
git branch -M main
git add .
git commit -m "Initial commit: Set up project and basic interface"
git remote add origin YOUR_GITHUB_REPOSITORY_URL
git push -u origin main
```

Replace YOUR_GITHUB_REPOSITORY_URL with the URL of your own repository.

If you already cloned the repository, do not repeat git init or git remote add origin. Work inside the cloned project folder instead.

Expected result: Your initial project files appear in your GitHub repository, and your first commit is visible in the commit history.

Step 4: Commit the Product List and Basic Interface

1. Create the product display section.

2. Add the six products and their prices.
3. Arrange the interface so that the product list and cart are easy to use.
4. Test the application.
5. Save your changes and execute:

```
git add .
git commit -m "Add product list and basic user interface"
git push
```

Expected result: GitHub records your product list and interface changes.

Step 5: Commit the Shopping Cart and Computation

1. Implement the Add to Cart functionality.
2. Add quantity controls and remove-item functionality.
3. Compute item subtotals and the overall total.
4. Test the calculations.
5. Execute:

```
git add .
git commit -m "Implement cart and order calculations"
git push
```

Expected result: A separate commit shows the implementation of the shopping cart and calculations.

Step 6: Commit Payment Validation and Receipt

1. Implement the cash payment input.
2. Validate blank, invalid, negative, and insufficient payment amounts.
3. Compute the correct change.
4. Generate the digital receipt.
5. Add the New Transaction functionality.
6. Test the complete transaction.
7. Execute:

```
git add .
git commit -m "Add payment validation and digital receipt"
git push
```

Expected result: GitHub records the payment, validation, receipt, and transaction improvements.

Step 7: Debug and Improve the Application

1. Review the application for errors.
2. Fix any issues with calculations, quantities, payment validation, or receipt generation.
3. Improve readability and usability.
4. Test the application again.

5. Execute:

```
git add .
git commit -m "Fix bugs and improve application"
git push
```

Expected result: Your repository contains a separate commit documenting debugging and final improvements.

Step 8: Verify Your GitHub Repository

1. Open your repository on GitHub.
2. Confirm that the latest source code is uploaded.
3. Open the commit history.
4. Verify that your commits appear in chronological order.
5. Confirm that your README describes the application and its basic features.
6. Copy the repository URL for submission.
7. Take a screenshot of your repository and commit history.

Important: A commit created locally does not appear on GitHub until it is successfully pushed. Check the GitHub website to confirm that your commits are visible.

V. AI-Assisted Development Requirements

You must use an AI-assisted development tool at least twice during the examination.

Document each AI interaction using the following format:

Required Information	Description
Prompt Used	The exact prompt submitted to the AI tool
Purpose	What you wanted the AI to accomplish
AI Response	A brief summary or screenshot of the response
Your Evaluation	Whether the response was correct, partially correct, or incorrect, and why
Modifications Made	Changes you made before using the generated output

Example Prompt 1:

"Create a simple shopping cart function for a Student Supplies Ordering System. Each item must have a name, unit price, and quantity. Automatically calculate the subtotal and total amount."

Example Prompt 2:

"Review my cash payment validation code. Identify errors that allow blank, negative, or insufficient payments, and suggest corrections. Explain the changes."

You must test the generated code and be able to explain how it works.

VI. Submission Requirements

Before the end of the 150-minute examination, submit the following:

1. **Working Application** – The complete project source code.
2. **GitHub Repository Link** – A link to your repository with visible commit history.
3. **AI Usage Documentation** – At least two documented AI prompts and their evaluations.
4. **Application Screenshots** – Evidence of the product list, shopping cart, payment validation, and digital receipt.
5. **GitHub Screenshot** – Evidence of your repository and commit history.
6. **README File** – A brief description of the application, its features, and instructions for running it.

VII. Final Checklist

Before submitting, make sure that:

- [ ] At least five products and their prices are displayed.
- [ ] Products can be added, removed, and adjusted in quantity.
- [ ] Subtotals and total amounts are calculated correctly.
- [ ] Payment validation works correctly.
- [ ] Change is calculated correctly.
- [ ] A digital receipt is generated after successful payment.
- [ ] A new transaction clears the previous transaction.
- [ ] At least two AI interactions are documented.
- [ ] The project is uploaded to GitHub.
- [ ] The required commits are visible on GitHub.
- [ ] The README and required screenshots are included.

Reminder: A simple application that works correctly is better than an elaborate application with incomplete functionality. Focus on meeting the requirements, testing your code, documenting AI assistance, and maintaining a clear GitHub commit history.